

LoadLead — Platform-Wide E2E Audit (v3)

Date: 2026-07-04 **Auditor:** Platform Engineering **Scope:** Full stack — backend business logic, frontend, and cross-environment parity (dev / staging / prod). Ref commit `c434c1c` (main).

1. Executive summary

The platform's **core business logic is healthy and well-tested**: 630/630 backend tests pass, frontend typechecks and builds clean, and the money-ledger, negotiation, and factoring seams hardened in the v1/v2 audits are intact with matching Global Secondary Indexes across prod and staging.

The material risk this round is **environment-parity drift**, not logic. A **Support/helpdesk feature that shipped straight to prod bypassed the environment-isolation system** entirely — its tables use prod-form default names in every environment, so a running staging (or dev) reads and writes **production** support data. Separately, **production silently records no membership audit logs** because that table was added to the module and staging/dev but never provisioned or configured in prod.

Verdict: Prod is functionally healthy and the transactional cores are sound. **1 HIGH** (cross-environment contamination), **1 MEDIUM** (prod audit-log gap), and **5 LOW** findings. None block prod today; the HIGH is latent and only fires when staging runs (currently paused → \$0).

Severity	Count	Headline
HIGH	1	Support tables resolve to prod in every env; boot-guard & parity-check blind to them
MEDIUM	1	Prod membership audit log silently dropped (missing table + dash-form default)
LOW	5	Dev unprovisioned · orphan prod table · parity-gate coverage gap · 963 KB FE bundle · beta-gate not runtime-observable

2. What was verified green

Area	Result
Backend unit/integration suite	78 files, 630 tests — all pass (the 4 known-stale fails from v1 are resolved)
Frontend typecheck (<code>tsc -b</code>)	clean
Frontend production build (<code>vite build</code>)	clean
Table env-var parity (the 50 <code>config.dynamodb</code> slots)	staging 50/50, dev 50/50 overridden

Area	Result
Money-ledger atomic idempotency (audit V2-H1)	intact — <code>attribute_not_exists</code> conditional writes present in reconciliation + funding
Money-table GSI queries (audit V2-M1)	intact — <code>invoiceId-index</code> used with scan fallback
Negotiation assign-release + agreed-rate binding	intact (<code>ensureAssignedAndReleased</code> , <code>agreedRate</code>)
GSI parity prod ↔ staging	perfect — <code>invoiceId-index</code> (Reconciliation, Funding), <code>loadId-createdAt-index</code> (LoadNegotiations), <code>negotiationId-createdAt-index</code> (NegotiationOffers) present in both
Prod live smoke	<code>/api/health</code> 200 · protected routes 401 · public support webhook 400 (correctly unauth) · frontend apex + beta 200
Beta config parity	prod <code>BETA_MODE=true</code> , staging <code>off</code> ; <code>APP_ENV</code> / <code>NODE_ENV</code> correct per env

Live table census: **prod 54 · staging 50 · dev 0**.

3. Findings

H1 — HIGH — Support feature reads/writes PROD tables from every environment

What. The Support/helpdesk service (`src/services/supportTicket.ts` , `src/routes/support.ts`) resolves its four tables from **non-standard env vars with prod-form defaults**:

```
process.env.SUPPORT_TICKETS_TABLE || 'LoadLead_SupportTickets'
process.env.SUPPORT_MESSAGES_TABLE || 'LoadLead_SupportMessages'
process.env.SUPPORT_INBOUND_TABLE || 'LoadLead_SupportInbound'
process.env.SUPPORT_SETTINGS_TABLE || 'LoadLead_SupportSettings'
```

These `SUPPORT*_TABLE` vars are **not** in `config/environment.ts` , **not** set in any env stack (`envs/staging` , `envs/dev` set zero of them), **not** in the Terraform `dynamodb_tableset` module, and **not** covered by `scripts/check-table-env-parity.mjs` . Consequently **every environment falls through to the `LoadLead_*` production defaults**.

Why it's dangerous. This is the exact cross-environment-contamination class the boot guard was built to stop after the v1 H1 finding — re-introduced by a feature that never went through the env-isolation system:

- The boot guard `assertTablesEnvIsolated()` inspects only `config.dynamodb` plus env vars explicitly present; Support is in neither, so it **cannot see the contamination** and staging boots clean.
- `POST /api/support/inbound` is a **public** webhook mounted *before* `authenticate` . A staging deploy processing an inbound email (or any Support admin action) writes tickets/messages into **prod's** `LoadLead_SupportTickets` .

Failure scenario. Start staging → an inbound support email (or a test webhook POST) hits `staging api /api/support/inbound` → a ticket is created in **production** `LoadLead_SupportTickets`, polluting prod support data and potentially triggering prod-side notifications.

Mitigation today. Staging is paused (\$0), so the path is not live right now. Risk activates the moment staging runs.

COA.

1. Move Support table resolution into `config/environment.ts` (`config.dynamodb.supportTickets`, ...) keyed on `DYNAMODB_SUPPORT_*_TABLE`; delete the inline prod-form defaults from `supportTicket.ts`.
2. Add the 4 Support tables to `modules/dynamodb_tableset` and to the staging/dev override maps; apply (staging tables ~\$0 on-demand).
3. Extend `check-table-env-parity.mjs` and `assertTablesEnvIsolated()` to cover the Support tables (see L3 for the systemic fix).
4. Until (1)–(3) land, do not run staging with Support reachable, or set `SUPPORT_*_TABLE` to the staging table names in the staging stack.

M1 — MEDIUM — Production records no membership audit logs

What. `OrgAuthService` (`src/services/orgService.ts`) resolves its table as:

```
const MEMBERSHIP_AUDIT_TABLE = process.env.DYNAMODB_MEMBERSHIP_AUDIT_TABLE
|| 'LoadLead-MembershipAuditLogs'; // dash form
```

- Prod sets **no** `DYNAMODB_MEMBERSHIP_AUDIT_TABLE` and has **no** `MembershipAuditLogs` table (live census: prod lacks it; staging has `LoadLead-Staging-MembershipAuditLogs`).
- So in prod the name falls to `LoadLead-MembershipAuditLogs` (**dash**, not the `LoadLead_` prod form) — a table that does not exist.
- The write is best-effort: `try { await Database.putItem(...) } catch (e) { Logger.error(...) }`. The membership operation succeeds; the **audit write fails and is swallowed**.

Impact. Production keeps **zero** membership audit trail (role changes, invites, removals). Staging works; prod is the odd one out — the IAM membership-audit table was added to the module + staging/dev but never provisioned or configured in prod. Compliance/audit-trail gap; also a latent instance of the dash-default bug family (cf. the `LoadLead-Drivers` default fixed in v1).

COA.

1. Provision `LoadLead_MembershipAuditLogs` in prod (add to the prod stack, apply) and set prod `DYNAMODB_MEMBERSHIP_AUDIT_TABLE=LoadLead_MembershipAuditLogs`.
2. Fix `orgService.ts` to resolve via `config.dynamodb.membershipAuditTable` (underscore prod form) instead of its own dash-form inline default.
3. Backfill is impossible (writes were dropped) — accept the gap start date; consider a metric/alarm so a silently-failing audit write is visible, not swallowed.

L1 — LOW — Dev environment is unprovisioned

Dev's Terraform stack and all 50 table overrides exist, but live dev has **0 DynamoDB tables** — the stack was never applied. "Parity" for dev is config-only; the environment is non-functional. **COA:** either apply dev (tables-only, on-demand ~\$0) so it's real, or retire the dev env from IaC/CI so it stops producing false parity signals.

L2 — LOW — Orphan `LoadLead_AdminAudit` table in prod

Code uses `LoadLead_AdminAuditLog` ; prod additionally carries a legacy `LoadLead_AdminAudit` (no "Log"), present in prod only and referenced nowhere in `src` . **COA:** confirm empty/unused, then delete (or import into IaC if intentional). Part of the 54-vs-50 count gap.

L3 — LOW — Parity gate gives false confidence (coverage gap)

`check-table-env-parity.mjs` validates only the 50 `config.dynamodb` slots and reported "parity passed" while the 4 Support tables (and any future out-of-band table) went unchecked. The gate that is supposed to prevent env drift **cannot detect the H1 class**. **COA:** make the check enumerate every `*_TABLE` env read in `src/` (AST or grep) rather than trusting `config.dynamodb` , or add a lint rule forbidding table-name resolution anywhere except `config/environment.ts` . This is the systemic fix that would have caught H1.

L4 — LOW — Frontend ships a single 963 KB JS bundle (no code-splitting)

`vite build` emits one `index-*.js` of **963 KB (265 KB gzip)** with a chunk-size warning. Slow first paint, especially on mobile / poor connections — relevant given the driver persona. **COA:** route-level `React.lazy()` + `dynamic import()` , and `build.rollupOptions.output.manualChunks` to split vendor (maps, charts) from app code.

L5 — LOW / informational — Beta gate is not runtime-observable

A nonexistent-user login returns identical `401 Invalid credentials` on **both** beta-on prod and beta-off staging, so there is no cheap runtime probe to confirm the gate is actually enforcing per environment (config parity is correct and the gate is unit-tested, but "is it live?" can't be checked from outside). **COA:** echo `betaMode` in `/api/health` (or a small `/api/beta/status`) so ops can verify live gate state per env — and so future audits have a real discriminator.

4. Course of action — prioritized

#	Action	Sev	Effort	Owner
1	Route Support tables through <code>config.dynamodb</code> + provision staging/dev tables + boot-guard/parity coverage	HIGH	M	Platform + owning team
2	Provision + wire prod <code>MembershipAuditLogs</code> ; fix dash-form default; alarm on swallowed audit writes	MED	S	Identity + Platform
3	Make <code>check-table-env-parity.mjs</code> enumerate all <code>*_TABLE</code> reads (systemic fix for H1 class)	LOW→systemic	S	Platform
4	Decide dev: provision (~\$0) or retire from IaC	LOW	S	Platform
5	Delete orphan <code>LoadLead_AdminAudit</code> after confirming unused	LOW	S	Platform

#	Action	Sev	Effort	Owner
6	Frontend code-splitting (route lazy + vendor chunks)	LOW	M	Marketplace
7	Add <code>betaMode</code> to <code>/api/health</code>	LOW	S	Platform

Sequencing: #1 and #3 together (the fix and the guard that prevents recurrence). #2 is quick and closes a compliance gap. #4–#7 are hygiene.

5. Recommendations (systemic)

1. **One source of truth for table names.** Nothing should resolve a DynamoDB table name outside `config/environment.ts`. The H1 and M1 findings both stem from services inventing their own `process.env.X || 'LoadLead...'` defaults. Enforce with a lint rule; it converts a whole bug family into a compile-time error.
 2. **Parity gate should discover, not trust.** The env-parity check should derive the table set from the code, not from a hand-maintained list that a new feature can side-step.
 3. **Prod is not automatically ahead of staging.** M1 shows a table that reached staging but not prod. Add a live "does every env have every table the code needs" probe (extends L3/L5) to the deploy pipeline, per environment.
 4. **Feature intake checklist.** Any new persisted store must: (a) land in `config.dynamodb`, (b) be added to the module + all env maps, (c) pass the parity check. The Support feature skipped all three.
-

6. Method & evidence

- Backend: `npm test` → 78 files / 630 tests pass.
 - Frontend: `tsc -b --noEmit + vite build` → clean.
 - Parity: `scripts/check-table-env-parity.mjs` (50/50); live `aws dynamodb list-tables` census per prefix; `aws elasticbeanstalk describe-configuration-settings` for `BETA_MODE/APP_ENV/NODE_ENV`; `describe-table` GSI comparison prod↔staging.
 - Contamination trace: `grep` of all `process.env.*_TABLE || 'LoadLead...'` defaults across `src/`, cross-referenced against `config/environment.ts` and the env stacks; confirmed only the 4 Support tables escape the system, and only `LoadLead-MembershipAuditLogs` uses a dash-form default.
 - Live smoke: prod `/api/health`, unauth guards, public support webhook, apex + beta frontends.
-

7. Correction (post-report, 2026-07-04)

M1 was partly a census artifact and is downgraded. The live-table census in §2/§3 grepped only `LoadLead_` (underscore) tables and **missed the dash-named `LoadLead-MembershipAuditLogs`**, which is **ACTIVE in prod with real data (18 records)**. So the claim "production records **no** membership audit logs" is **incorrect** — prod audit logging works, via the dash-named table. Prod actually has **55** tables (54 `LoadLead_` + 1 `LoadLead-MembershipAuditLogs`), not 54.

What remains true: the value bypassed `config.dynamodb` (fixed in PR #28), and the name is **inconsistent** with the `LoadLead_` convention (a cosmetic wart, not a functional bug). Revised severity: **LOW (naming inconsistency)**, not MEDIUM.

Note: the M1 fix as first merged (PR #28) defaulted to the underscore form and was a **latent regression** (would have broken prod on deploy). Corrected in **PR #29** — the default is now the real `LoadLead-MembershipAuditLogs`; the config-routing/coverage from #28 is retained; no prod table provisioning is needed. **H1 (Support) is unaffected and stands as reported.**

Process lesson: a live-table census must match **both** separators (`LoadLead_` and `LoadLead-`). The systemic recommendation (§5) stands and would also cover this: table names should be discovered from code/config, not from ad-hoc `list-tables` greps.